

UNIVERSITATEA DIN BUCUREȘTI

**FACULTATEA DE
MATEMATICĂ ȘI
INFORMATICĂ**



Master BDTS IFR

Securitatea bazelor de date

Gestionarea unei firme de construcții

Mîndruleanu Matei Daniel

Mai 2026

Introducere

a. Prezentarea succintă a modelului proiectat și a regulilor sale

Pentru realizarea acestui proiect a fost utilizată schema bazei de date dezvoltată anterior în cadrul proiectelor disciplinelor Data Warehousing & Business Intelligence și Metode de optimizare și distribuire în baze de date.

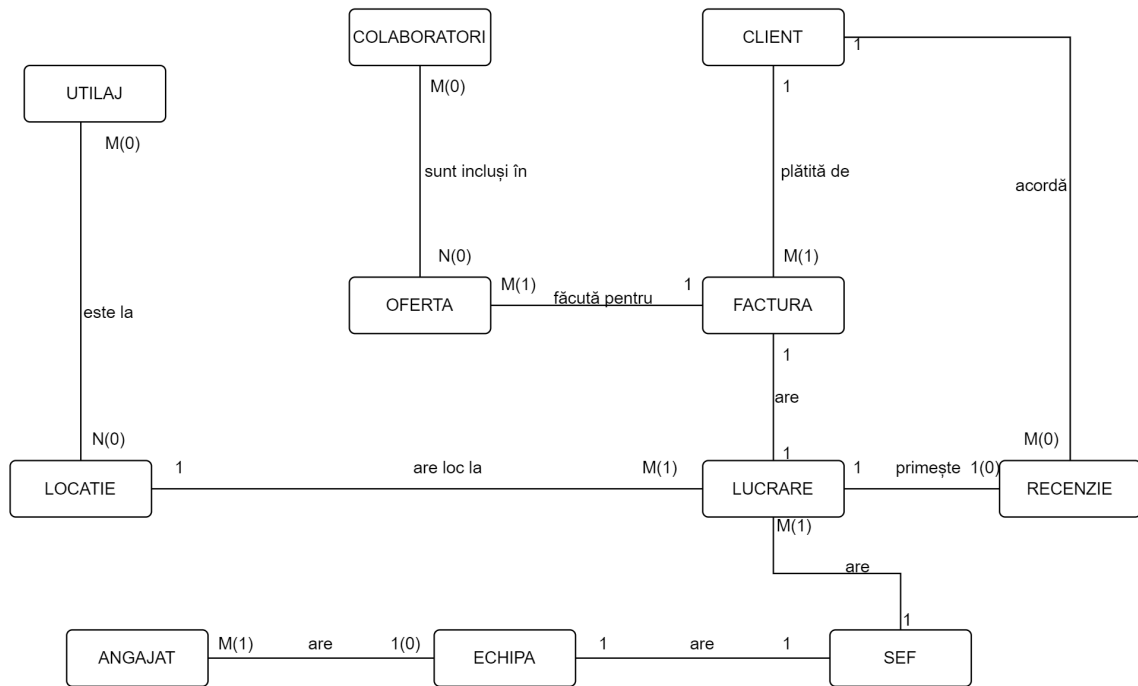
Modelul proiectat are ca scop gestionarea activităților desfășurate în cadrul unei firme de construcții, prin organizarea și administrarea informațiilor referitoare la clienți, lucrări, locații, echipe de lucru, angajați, șefi de echipă, utilaje, colaboratori și documentele asociate acestora.

Structura bazei de date este realizată utilizând modelul relațional și este alcătuită din mai multe entități interconectate prin relații de tip one-to-one, one-to-many și many-to-many. Aceste relații permit reprezentarea corectă a proceselor specifice domeniului și asigură consistența informațiilor stocate.

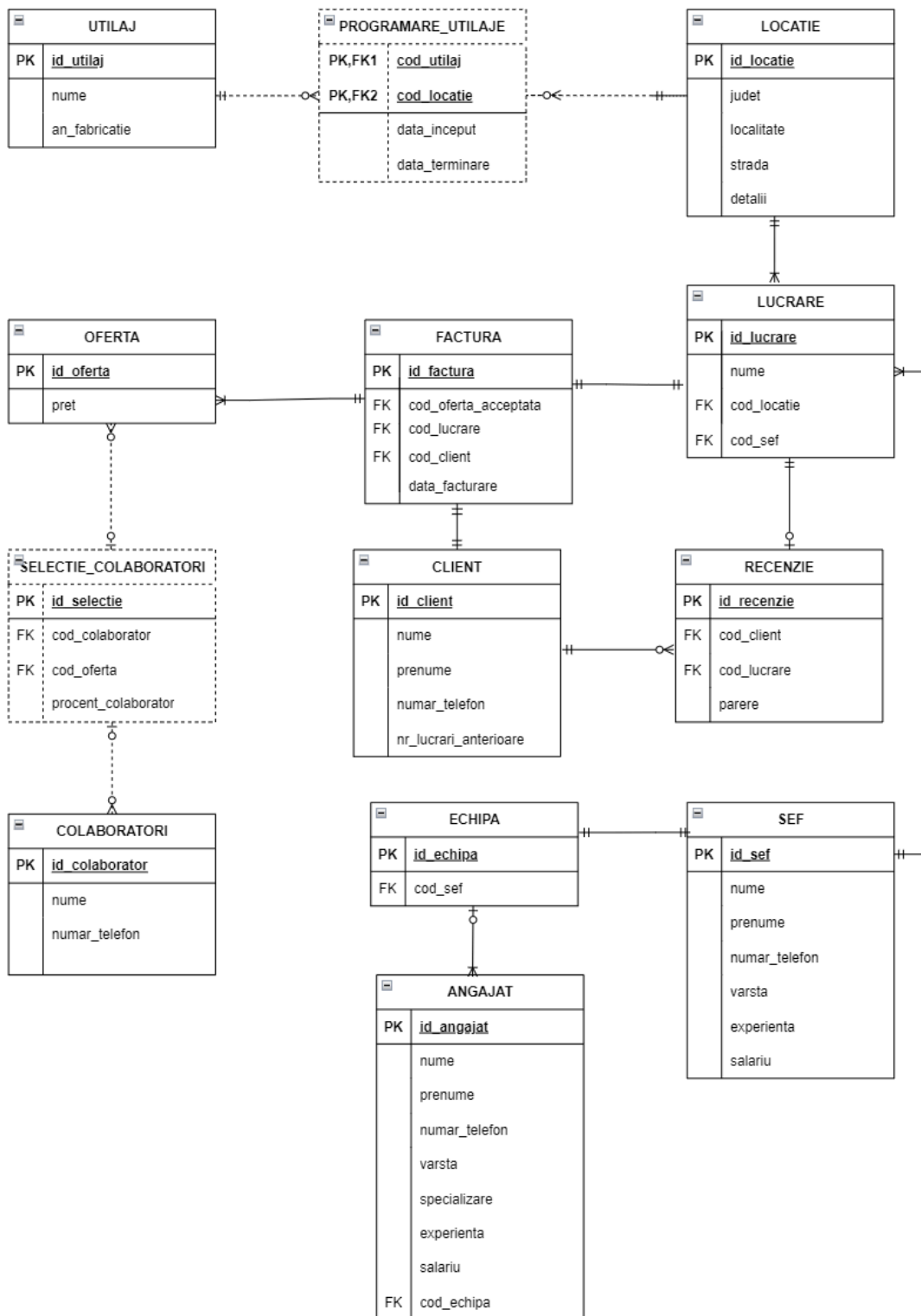
Un client poate solicita realizarea uneia sau mai multor lucrări, fiecare lucrare fiind asociată unei locații și coordonată de un șef de echipă. Pentru executarea lucrărilor sunt utilizate echipe formate din angajați specializați, precum și diferite utilaje programate în funcție de necesitățile proiectului. De asemenea, firma poate colabora cu parteneri externi implicați în procesul de execuție prin intermediul ofertelor și al contractelor de colaborare.

Modelul implementează reguli de integritate prin utilizarea cheilor primare și a cheilor externe, astfel încât fiecare înregistrare să poată fi identificată în mod unic și să fie menținute corect relațiile dintre entități. În plus, sunt definite constrângeri de validare pentru a preveni introducerea unor valori necorespunzătoare, cum ar fi vârste negative, salarii invalide sau referințe către entități inexistente.

b. Diagrama conceptuală



c. Schema relațională



d. Crearea tabelelor (script separat)

Pentru început, am creat un utilizator nou din admin specific pentru acest proiect.

```
-- CREARE USER SPECIAL PT PROIECT
CREATE USER USER_SECURITATE
IDENTIFIED BY 1234;

-- ACORDAM DREPTURI
GRANT CONNECT, RESOURCE TO USER_SECURITATE;

ALTER USER USER_SECURITATE
QUOTA UNLIMITED ON USERS;

GRANT CREATE VIEW TO USER_SECURITATE;
GRANT CREATE SYNONYM TO USER_SECURITATE;

--pt criptare
GRANT EXECUTE ON SYS.DBMS_CRYPTO TO USER_SECURITATE;

--pt auditare
GRANT EXECUTE ON SYS.DBMS_FGA TO USER_SECURITATE;
```

Creare tabele

```
--CREATE TABLES

-- LOCATIE
CREATE TABLE LOCATIE (
    id_locatie NUMBER,
    judet VARCHAR2(20),
    localitate VARCHAR2(20),
    strada VARCHAR2(20),
    detalii VARCHAR2(50),

    CONSTRAINT pk_locatie
        PRIMARY KEY (id_locatie)
);

-- UTILAJ
CREATE TABLE UTILAJ (
    id_utilaj NUMBER,
    nume VARCHAR2(50) NOT NULL,
    an_fabricatie DATE NOT NULL,

    CONSTRAINT pk_utilaj
        PRIMARY KEY (id_utilaj)
```

```

);

-- COLABORATORI
CREATE TABLE COLABORATORI (
    id_colaborator NUMBER,
    nume VARCHAR2(100) NOT NULL,
    numar_telefon VARCHAR2(15) NOT NULL,

    CONSTRAINT pk_colaboratori
        PRIMARY KEY (id_colaborator)
);

-- CLIENT
CREATE TABLE CLIENT (
    id_client NUMBER,
    nume VARCHAR2(100) NOT NULL,
    prenume VARCHAR2(100) NOT NULL,
    numar_telefon VARCHAR2(15) NOT NULL,
    nr_lucrari_anterioare NUMBER,

    CONSTRAINT pk_client
        PRIMARY KEY (id_client),

    CONSTRAINT ck_client_lucrari
        CHECK (nr_lucrari_anterioare >= 0)
);

-- OFERTA
CREATE TABLE OFERTA (
    id_oferta NUMBER,
    pret NUMBER,

    CONSTRAINT pk_oferta
        PRIMARY KEY (id_oferta)
);

-- SEF
CREATE TABLE SEF (
    id_sef NUMBER,
    nume VARCHAR2(100) NOT NULL,
    prenume VARCHAR2(100) NOT NULL,
    numar_telefon VARCHAR2(15) NOT NULL,
    varsta NUMBER,
    experienta NUMBER,
    salariu NUMBER,

    CONSTRAINT pk_sef
        PRIMARY KEY (id_sef),

```

```

CONSTRAINT ck_sef_varsta
    CHECK (varsta >= 18),

CONSTRAINT ck_sef_experienta
    CHECK (experienta >= 0),

CONSTRAINT ck_sef_salariu
    CHECK (salariu > 0)
);

-- LUCRARE
CREATE TABLE LUCRARE (
    id_lucrare NUMBER,
    nume VARCHAR2(50),
    cod_locatie NUMBER NOT NULL,
    cod_sef NUMBER NOT NULL,

    CONSTRAINT pk_lucrare
        PRIMARY KEY (id_lucrare),

    CONSTRAINT fk_lucrare_locatie
        FOREIGN KEY (cod_locatie)
        REFERENCES LOCATIE(id_locatie),

    CONSTRAINT fk_lucrare_sef
        FOREIGN KEY (cod_sef)
        REFERENCES SEF(id_sef)
);

-- FACTURA
CREATE TABLE FACTURA (
    id_factura NUMBER,
    cod_oferta_acceptata NUMBER,
    cod_client NUMBER,
    cod_lucrare NUMBER NOT NULL,
    data_facturare DATE NOT NULL,

    CONSTRAINT pk_factura
        PRIMARY KEY (id_factura),

    CONSTRAINT fk_factura_oferta
        FOREIGN KEY (cod_oferta_acceptata)
        REFERENCES OFERTA(id_oferta),

    CONSTRAINT fk_factura_client
        FOREIGN KEY (cod_client)
        REFERENCES CLIENT(id_client),

```

```

        CONSTRAINT fk_factura_lucrare
            FOREIGN KEY (cod_lucrare)
            REFERENCES LUCRARE(id_lucrare)
    );

-- RECENZIE
CREATE TABLE RECENZIE (
    id_recenzie NUMBER,
    cod_client NUMBER NOT NULL,
    cod_lucrare NUMBER NOT NULL,
    parere VARCHAR2(200),

    CONSTRAINT pk_recenzie
        PRIMARY KEY (id_recenzie),

    CONSTRAINT fk_recenzie_client
        FOREIGN KEY (cod_client)
        REFERENCES CLIENT(id_client),

    CONSTRAINT fk_recenzie_lucrare
        FOREIGN KEY (cod_lucrare)
        REFERENCES LUCRARE(id_lucrare)
);

-- ECHIPA
CREATE TABLE ECHIPA (
    id echipa NUMBER,
    cod_sef NUMBER NOT NULL,

    CONSTRAINT pk_echipa
        PRIMARY KEY (id_echipa),

    CONSTRAINT fk_echipa_sef
        FOREIGN KEY (cod_sef)
        REFERENCES SEF(id_sef)
);

-- ANGAJAT
CREATE TABLE ANGAJAT (
    id_angajat NUMBER,
    nume VARCHAR2(100) NOT NULL,
    prenume VARCHAR2(100) NOT NULL,
    numar_telefon VARCHAR2(15) NOT NULL,
    varsta NUMBER,
    specializare VARCHAR2(30),
    experienta NUMBER,
    salariu NUMBER,
    cod_echipa NUMBER,

```



```

CONSTRAINT pk_angajat
    PRIMARY KEY (id_angajat),

CONSTRAINT ck_angajat_varsta
    CHECK (varsta >= 18),

CONSTRAINT ck_angajat_experienta
    CHECK (experienta >= 0),

CONSTRAINT ck_angajat_salariu
    CHECK (salariu > 0),

CONSTRAINT fk_angajat echipa
    FOREIGN KEY (cod echipa)
    REFERENCES ECHIPA(id echipa)
);

-- PROGRAMARE_UTILAJE
CREATE TABLE PROGRAMARE_UTILAJE (
    cod_utilaj NUMBER NOT NULL,
    cod_locatie NUMBER NOT NULL,
    data_inceput DATE NOT NULL,
    data_terminare DATE NOT NULL,

    CONSTRAINT pk_programare_utilaje
        PRIMARY KEY (cod_utilaj, cod_locatie),

    CONSTRAINT fk_programare_utilaj
        FOREIGN KEY (cod_utilaj)
        REFERENCES UTILAJ(id_utilaj),

    CONSTRAINT fk_programare_locatie
        FOREIGN KEY (cod_locatie)
        REFERENCES LOCATIE(id_locatie)
);

-- SELECTIE_COLABORATORI
CREATE TABLE SELECTIE_COLABORATORI (
    id_selectie NUMBER,
    cod_colaborator NUMBER NOT NULL,
    cod_oferta NUMBER NOT NULL,
    procent_colaborator NUMBER DEFAULT 10,

    CONSTRAINT pk_selectie_colaboratori
        PRIMARY KEY (id_selectie),

    CONSTRAINT fk_selectie_colaborator
        FOREIGN KEY (cod_colaborator)

```

```

REFERENCES COLABORATORI(id_colaborator),

CONSTRAINT fk_selectie_oferta
    FOREIGN KEY (cod_oferta)
    REFERENCES OFERTA(id_oferta)
);

```

Populare tabele

--insert data into tables

-- LOCATIE

```

INSERT INTO LOCATIE VALUES (1,'Bucuresti','Sector 1','Aviatorilor','Bloc A1');
INSERT INTO LOCATIE VALUES (2,'Bucuresti','Sector 3','Decebal','Bloc B2');
INSERT INTO LOCATIE VALUES (3,'Cluj','Cluj-Napoca','Memorandumului','Casa 12');
INSERT INTO LOCATIE VALUES (4,'Timis','Timisoara','Take Ionescu','Birou 4');
INSERT INTO LOCATIE VALUES (5,'Iasi','Iasi','Pacurari','Vila 5');
INSERT INTO LOCATIE VALUES (6,'Brasov','Brasov','Muresenilor','Etaj 2');
INSERT INTO LOCATIE VALUES (7,'Constanta','Constanta','Mamaia','Hotel 3');
INSERT INTO LOCATIE VALUES (8,'Sibiu','Sibiu','Nicolae Balcescu','Spatiu
comercial');
INSERT INTO LOCATIE VALUES (9,'Bihor','Oradea','Republicii','Bloc C3');
INSERT INTO LOCATIE VALUES (10,'Arges','Pitesti','Exercitiu','Apartament 10');

```

-- UTILAJ

```

INSERT INTO UTILAJ VALUES (1,'Excavator CAT', DATE '2018-05-10');
INSERT INTO UTILAJ VALUES (2,'Buldozer Komatsu', DATE '2019-03-12');
INSERT INTO UTILAJ VALUES (3,'Macara Liebherr', DATE '2020-06-15');
INSERT INTO UTILAJ VALUES (4,'Miniincarcator Bobcat', DATE '2017-02-01');
INSERT INTO UTILAJ VALUES (5,'Excavator Volvo', DATE '2021-08-20');
INSERT INTO UTILAJ VALUES (6,'Buldoexcavator JCB', DATE '2022-04-18');
INSERT INTO UTILAJ VALUES (7,'Compactor Hamm', DATE '2020-10-05');
INSERT INTO UTILAJ VALUES (8,'Autobetoniera MAN', DATE '2019-09-11');
INSERT INTO UTILAJ VALUES (9,'Macara Grove', DATE '2023-01-15');
INSERT INTO UTILAJ VALUES (10,'Incarcator frontal CAT', DATE '2024-02-01');

```

-- COLABORATORI

```

INSERT INTO COLABORATORI VALUES (1,'Construct Expert SRL','0711111111');
INSERT INTO COLABORATORI VALUES (2,'Mega Build SRL','0722222222');
INSERT INTO COLABORATORI VALUES (3,'Urban Construct','0733333333');
INSERT INTO COLABORATORI VALUES (4,'Smart Engineering','0744444444');
INSERT INTO COLABORATORI VALUES (5,'Delta Construct','0755555555');
INSERT INTO COLABORATORI VALUES (6,'Nord Vest Grup','0766666666');
INSERT INTO COLABORATORI VALUES (7,'Proiect Plus','0777777777');
INSERT INTO COLABORATORI VALUES (8,'Arhitect Design','0788888888');
INSERT INTO COLABORATORI VALUES (9,'Construct Rapid','0799999999');
INSERT INTO COLABORATORI VALUES (10,'Build Team','0700000000');

```

-- CLIENT

```
INSERT INTO CLIENT VALUES (1, 'Popescu', 'Andrei', '0712000001', 2);
INSERT INTO CLIENT VALUES (2, 'Ionescu', 'Maria', '0712000002', 0);
INSERT INTO CLIENT VALUES (3, 'Dumitru', 'Alex', '0712000003', 1);
INSERT INTO CLIENT VALUES (4, 'Georgescu', 'Elena', '0712000004', 4);
INSERT INTO CLIENT VALUES (5, 'Stan', 'Mihai', '0712000005', 3);
INSERT INTO CLIENT VALUES (6, 'Radu', 'Cristina', '0712000006', 0);
INSERT INTO CLIENT VALUES (7, 'Voicu', 'Daniel', '0712000007', 5);
INSERT INTO CLIENT VALUES (8, 'Marin', 'Ana', '0712000008', 2);
INSERT INTO CLIENT VALUES (9, 'Enache', 'Paul', '0712000009', 1);
INSERT INTO CLIENT VALUES (10, 'Ilie', 'Raluca', '0712000010', 6);
```

-- OFERTA

```
INSERT INTO OFERTA VALUES (1, 15000);
INSERT INTO OFERTA VALUES (2, 28000);
INSERT INTO OFERTA VALUES (3, 45000);
INSERT INTO OFERTA VALUES (4, 32000);
INSERT INTO OFERTA VALUES (5, 67000);
INSERT INTO OFERTA VALUES (6, 52000);
INSERT INTO OFERTA VALUES (7, 84000);
INSERT INTO OFERTA VALUES (8, 21000);
INSERT INTO OFERTA VALUES (9, 39000);
INSERT INTO OFERTA VALUES (10, 96000);
```

-- SEF

```
INSERT INTO SEF VALUES (1, 'Popa', 'Andrei', '0723000001', 42, 15, 8500);
INSERT INTO SEF VALUES (2, 'Ionescu', 'Mihai', '0723000002', 39, 12, 7800);
INSERT INTO SEF VALUES (3, 'Georgescu', 'Elena', '0723000003', 45, 20, 9200);
INSERT INTO SEF VALUES (4, 'Marin', 'Cristian', '0723000004', 38, 11, 7600);
INSERT INTO SEF VALUES (5, 'Stan', 'Raluca', '0723000005', 41, 14, 8700);
INSERT INTO SEF VALUES (6, 'Dumitru', 'Paul', '0723000006', 50, 25, 10500);
INSERT INTO SEF VALUES (7, 'Voicu', 'Alexandru', '0723000007', 37, 10, 7400);
INSERT INTO SEF VALUES (8, 'Enache', 'Maria', '0723000008', 43, 17, 9000);
INSERT INTO SEF VALUES (9, 'Ilie', 'Cosmin', '0723000009', 40, 13, 8200);
INSERT INTO SEF VALUES (10, 'Radu', 'Ana', '0723000010', 36, 9, 7000);
```

-- ECHIPA

```
INSERT INTO ECHIPA VALUES (1, 1);
INSERT INTO ECHIPA VALUES (2, 2);
INSERT INTO ECHIPA VALUES (3, 3);
INSERT INTO ECHIPA VALUES (4, 4);
INSERT INTO ECHIPA VALUES (5, 5);
INSERT INTO ECHIPA VALUES (6, 6);
INSERT INTO ECHIPA VALUES (7, 7);
```

```
INSERT INTO ECHIPA VALUES (8,8);
INSERT INTO ECHIPA VALUES (9,9);
INSERT INTO ECHIPA VALUES (10,10);
```

-- ANGAJAT

```
INSERT INTO ANGAJAT VALUES
(1,'Matei','Daniel','0734000001',24,'Zidar',2,4500,1);
INSERT INTO ANGAJAT VALUES
(2,'Popescu','Andrei','0734000002',29,'Electrician',5,5200,2);
INSERT INTO ANGAJAT VALUES
(3,'Ionescu','Maria','0734000003',27,'Instalator',4,5000,3);
INSERT INTO ANGAJAT VALUES
(4,'Stan','Cristian','0734000004',31,'Finisor',7,5600,4);
INSERT INTO ANGAJAT VALUES
(5,'Voicu','Elena','0734000005',26,'Dulgher',3,4700,5);
INSERT INTO ANGAJAT VALUES (6,'Marin','Paul','0734000006',28,'Operator
utilaj',5,5500,6);
INSERT INTO ANGAJAT VALUES (7,'Radu','Alex','0734000007',35,'Sudor',10,6500,7);
INSERT INTO ANGAJAT VALUES
(8,'Enache','Ana','0734000008',30,'Arhitect',6,6200,8);
INSERT INTO ANGAJAT VALUES
(9,'Ilie','George','0734000009',25,'Topograf',2,4800,9);
INSERT INTO ANGAJAT VALUES
(10,'Dumitru','Raluca','0734000010',33,'Inginer',8,7000,10);
```

-- LUCRARE

```
INSERT INTO LUCRARE VALUES (1,'Fundatie casa',1,1);
INSERT INTO LUCRARE VALUES (2,'Bloc locuinte',2,2);
INSERT INTO LUCRARE VALUES (3,'Renovare cladire',3,3);
INSERT INTO LUCRARE VALUES (4,'Amenajare birouri',4,4);
INSERT INTO LUCRARE VALUES (5,'Constructie hala',5,5);
INSERT INTO LUCRARE VALUES (6,'Extindere depozit',6,6);
INSERT INTO LUCRARE VALUES (7,'Reabilitare fatada',7,7);
INSERT INTO LUCRARE VALUES (8,'Parcare subterana',8,8);
INSERT INTO LUCRARE VALUES (9,'Complex rezidential',9,9);
INSERT INTO LUCRARE VALUES (10,'Constructie vila',10,10);
```

-- FACTURA

```
INSERT INTO FACTURA VALUES (1,1,1,1,DATE '2024-01-15');
INSERT INTO FACTURA VALUES (2,2,2,2,DATE '2024-02-10');
INSERT INTO FACTURA VALUES (3,3,3,3,DATE '2024-03-05');
INSERT INTO FACTURA VALUES (4,4,4,4,DATE '2024-04-12');
INSERT INTO FACTURA VALUES (5,5,5,5,DATE '2024-05-20');
INSERT INTO FACTURA VALUES (6,6,6,6,DATE '2024-06-08');
INSERT INTO FACTURA VALUES (7,7,7,7,DATE '2024-07-18');
INSERT INTO FACTURA VALUES (8,8,8,8,DATE '2024-08-03');
```

```
INSERT INTO FACTURA VALUES (9,9,9,9,DATE '2024-09-14');
INSERT INTO FACTURA VALUES (10,10,10,10,DATE '2024-10-01');
```

-- RECENZIE

```
INSERT INTO RECENZIE VALUES (1,1,1,'Lucrare executata foarte bine.');
```

```
INSERT INTO RECENZIE VALUES (2,2,2,'Personal profesionist.');
```

```
INSERT INTO RECENZIE VALUES (3,3,3,'Calitate buna a serviciilor.');
```

```
INSERT INTO RECENZIE VALUES (4,4,4,'Respectarea termenelor stabilite.');
```

```
INSERT INTO RECENZIE VALUES (5,5,5,'Pret corect pentru serviciile oferite.');
```

```
INSERT INTO RECENZIE VALUES (6,6,6,'Comunicare foarte buna.');
```

```
INSERT INTO RECENZIE VALUES (7,7,7,'Echipa bine organizata.');
```

```
INSERT INTO RECENZIE VALUES (8,8,8,'Rezultat peste asteptari.');
```

```
INSERT INTO RECENZIE VALUES (9,9,9,'Recomand cu incredere.');
```

```
INSERT INTO RECENZIE VALUES (10,10,10,'Foarte multumit de colaborare.');
```

-- PROGRAMARE_UTILAJE

```
INSERT INTO PROGRAMARE_UTILAJE VALUES (1,1,DATE '2024-01-01',DATE '2024-01-20');
```

```
INSERT INTO PROGRAMARE_UTILAJE VALUES (2,2,DATE '2024-02-01',DATE '2024-02-25');
```

```
INSERT INTO PROGRAMARE_UTILAJE VALUES (3,3,DATE '2024-03-01',DATE '2024-03-30');
```

```
INSERT INTO PROGRAMARE_UTILAJE VALUES (4,4,DATE '2024-04-01',DATE '2024-04-18');
```

```
INSERT INTO PROGRAMARE_UTILAJE VALUES (5,5,DATE '2024-05-01',DATE '2024-05-22');
```

```
INSERT INTO PROGRAMARE_UTILAJE VALUES (6,6,DATE '2024-06-01',DATE '2024-06-28');
```

```
INSERT INTO PROGRAMARE_UTILAJE VALUES (7,7,DATE '2024-07-01',DATE '2024-07-19');
```

```
INSERT INTO PROGRAMARE_UTILAJE VALUES (8,8,DATE '2024-08-01',DATE '2024-08-26');
```

```
INSERT INTO PROGRAMARE_UTILAJE VALUES (9,9,DATE '2024-09-01',DATE '2024-09-23');
```

```
INSERT INTO PROGRAMARE_UTILAJE VALUES (10,10,DATE '2024-10-01',DATE '2024-10-31');
```

-- SELECTIE_COLABORATORI

```
INSERT INTO SELECTIE_COLABORATORI VALUES (1,1,1,10);
```

```
INSERT INTO SELECTIE_COLABORATORI VALUES (2,2,2,12);
```

```
INSERT INTO SELECTIE_COLABORATORI VALUES (3,3,3,15);
```

```
INSERT INTO SELECTIE_COLABORATORI VALUES (4,4,4,8);
```

```
INSERT INTO SELECTIE_COLABORATORI VALUES (5,5,5,20);
```

```
INSERT INTO SELECTIE_COLABORATORI VALUES (6,6,6,18);
```

```
INSERT INTO SELECTIE_COLABORATORI VALUES (7,7,7,10);
```

```
INSERT INTO SELECTIE_COLABORATORI VALUES (8,8,8,25);
```

```
INSERT INTO SELECTIE_COLABORATORI VALUES (9,9,9,14);  
INSERT INTO SELECTIE_COLABORATORI VALUES (10,10,10,30);
```

e. Prezentarea regulilor de securitate care vor fi aplicate asupra modelului

În cadrul aplicației au fost definite mai multe mecanisme de securitate pentru protejarea informațiilor gestionate de baza de date. Aceste mecanisme urmăresc asigurarea confidențialității, integrității și trasabilității operațiilor efectuate asupra datelor.

Principalele reguli de securitate aplicate sunt:

- protejarea informațiilor sensibile prin criptarea datelor
- controlul accesului pe baza utilizatorilor și a privilegiilor acordate
- separarea responsabilităților între utilizatorii sistemului
- auditarea operațiilor efectuate asupra datelor prin mecanisme standard Oracle și prin auditare personalizată
- monitorizarea accesului la informațiile sensibile prin politici Fine Grained Auditing (FGA)
- limitarea accesului la funcțiile de criptare și decriptare doar utilizatorilor autorizați

Criptarea Datelor

Una dintre măsurile de securitate implementate în cadrul proiectului este protejarea datelor sensibile prin criptare. Pentru această etapă a fost utilizat pachetul DBMS_CRYPTO pus la dispoziție de Oracle, conform conceptelor prezentate în cadrul laboratoarelor.

În cadrul proiectului au fost selectate pentru protecție anumite informații considerate sensibile din tabela ANGAJAT, respectiv numărul de telefon și salariul angajaților. Scopul implementării este de a demonstra modul în care datele pot fi stocate într-o formă criptată și ulterior recuperate prin decriptare folosind aceeași cheie.

Pentru realizarea criptării a fost utilizat algoritmul AES-128. În scop demonstrativ, valorile criptate sunt stocate în coloane suplimentare ale tabelului ANGAJAT, păstrând în paralel și valorile originale pentru a putea evidenția mai ușor procesul de criptare și decriptare.

```
--criptarea datelor
```

```
--modificare tabel angajat || agaudarea coloanelor suplimentare pentru datele criptate
```

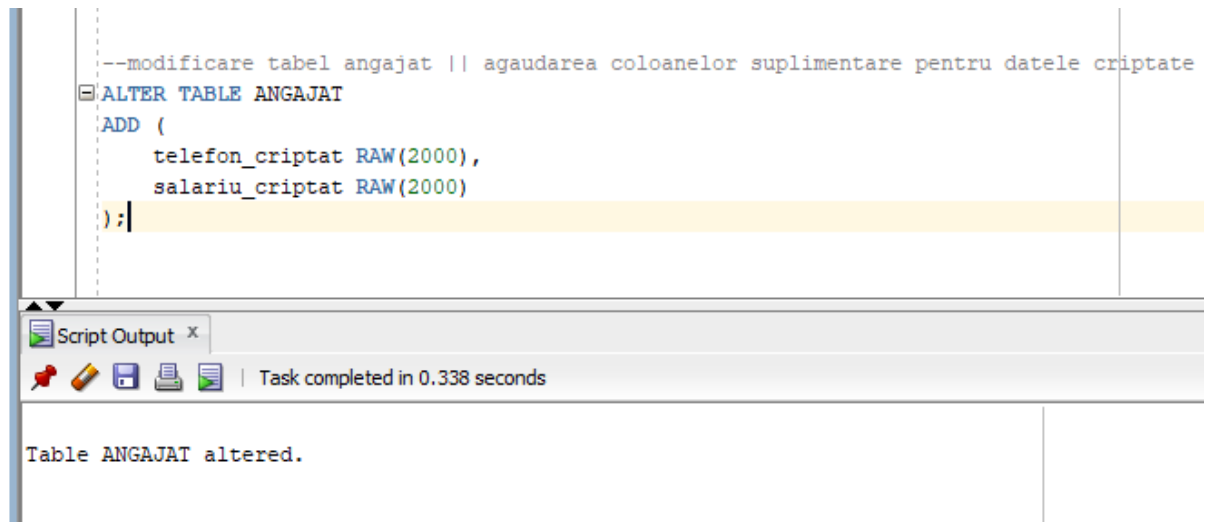
```
ALTER TABLE ANGAJAT
```

```
ADD (
```

```
    telefon_criptat RAW(2000),
```

```
    salariu_criptat RAW(2000)
```

```
);
```



```
-- Functie pentru criptarea unui text folosind AES128
```

```
CREATE OR REPLACE FUNCTION criptare_text(
```

```
    p_text VARCHAR2          -- textul care trebuie criptat
```

```
)
```

```
RETURN RAW                  -- rezultatul va fi binar (RAW)
```

```
IS
```

```
    -- cheia folosita pentru criptare
```

```
    -- AES128 necesita o cheie de 16 bytes (128 biti) = 16 caractere in string
```

```
    l_key RAW(16) :=
```

```
        UTL_RAW.CAST_TO_RAW('BDTS123456789101');
```

```
BEGIN
```

```
    -- DBMS_CRYPTO lucreaza doar cu date de tip RAW,
```

```
    -- de aceea transformam textul primit in RAW
```

```

RETURN DBMS_CRYPTO.ENCRYPT(
    src => UTL_I18N.STRING_TO_RAW(
        p_text,
        'AL32UTF8'
    ),
    -- algoritmul utilizat:
    -- AES128 = algoritm de criptare
    -- CHAIN_CBC = modul CBC
    -- PAD_PKCS5 = completare automata a ultimului bloc
    typ => DBMS_CRYPTO.ENCRYPT_AES128
        + DBMS_CRYPTO.CHAIN_CBC
        + DBMS_CRYPTO.PAD_PKCS5,

    -- cheia folosita la criptare
    key => l_key
);
END;
/

```



```

-- Functie pentru criptarea unui text folosind AES128
CREATE OR REPLACE FUNCTION criptare_text(
    p_text VARCHAR2          -- textul care trebuie criptat
)
RETURN RAW                  -- rezultatul va fi binar (RAW)
IS
    -- cheia folosita pentru criptare
    -- AES128 necesita o cheie de 16 bytes (128 biti) = 16 caractere in string
    l_key RAW(16) :=
        UTL_RAW.CAST_TO_RAW('BDTS123456789101');

BEGIN
    -- DBMS_CRYPTO lucreaza doar cu date de tip RAW,
    -- de aceea transformam textul primit in RAW
    RETURN DBMS_CRYPTO.ENCRYPT(
        src => UTL_I18N.STRING_TO_RAW(
            p_text,
            'AL32UTF8'
        ),
        -- algoritmul utilizat:
        -- AES128 = algoritm de criptare
        -- CHAIN_CBC = modul CBC
        -- PAD_PKCS5 = completare automata a ultimului bloc
        typ => DBMS_CRYPTO.ENCRYPT_AES128
            + DBMS_CRYPTO.CHAIN_CBC
            + DBMS_CRYPTO.PAD_PKCS5,
        -- cheia folosita la criptare
        key => l_key
    );
END;
/

```

Script Output x

Task completed in 0.472 seconds

Function CRIPTARE_TEXT compiled

```

-- Functie pentru decriptarea valorilor criptate cu AES128
CREATE OR REPLACE FUNCTION decriptare_text(
    p_raw RAW
)
RETURN VARCHAR2
IS
    -- aceeaasi cheie folosita la criptare
    l_key RAW(16) :=
        UTL_RAW.CAST_TO_RAW('BDTS123456789101');

    l_decrypted RAW(2000);

BEGIN
    -- decriptarea valorii
    l_decrypted :=
        DBMS_CRYPTO.DECRYPT(
            src => p_raw,
            typ => DBMS_CRYPTO.ENCRYPT_AES128
                + DBMS_CRYPTO.CHAIN_CBC
                + DBMS_CRYPTO.PAD_PKCS5,
            key => l_key
        );
    -- transformarea rezultatului RAW in text
    RETURN UTL_I18N.RAW_TO_CHAR(
        l_decrypted,
        'AL32UTF8'
    );
END;
/

```

```
-- Functie pentru decriptarea valorilor criptate cu AES128
CREATE OR REPLACE FUNCTION decriptare_text(
  p_raw RAW
)
RETURN VARCHAR2
IS
  -- aceeasi cheie folosita la criptare
  l_key RAW(16) :=
    UTL_RAW.CAST_TO_RAW('BDTS123456789101');

  l_decrypted RAW(2000);

BEGIN
  -- decriptarea valorii
  l_decrypted :=
    DBMS_CRYPTO.DECRYPT(
      src => p_raw,
      typ => DBMS_CRYPTO.ENCRYPT_AES128
        + DBMS_CRYPTO.CHAIN_CBC
        + DBMS_CRYPTO.PAD_PKCS5,
      key => l_key
    );
  -- transformarea rezultatului RAW in text
  RETURN UTL_I18N.RAW_TO_CHAR(
    l_decrypted,
    'AL32UTF8'
  );
END;
```

Script Output x Query Result x

Task completed in 0.068 seconds

Function DECRYPTARE_TEXT compiled

Demonstrate criptare și decriptare pe un singur angajat

```
SELECT *
FROM ANGAJAT
WHERE id_angajat = 1;
```

Script Output x Query Result x

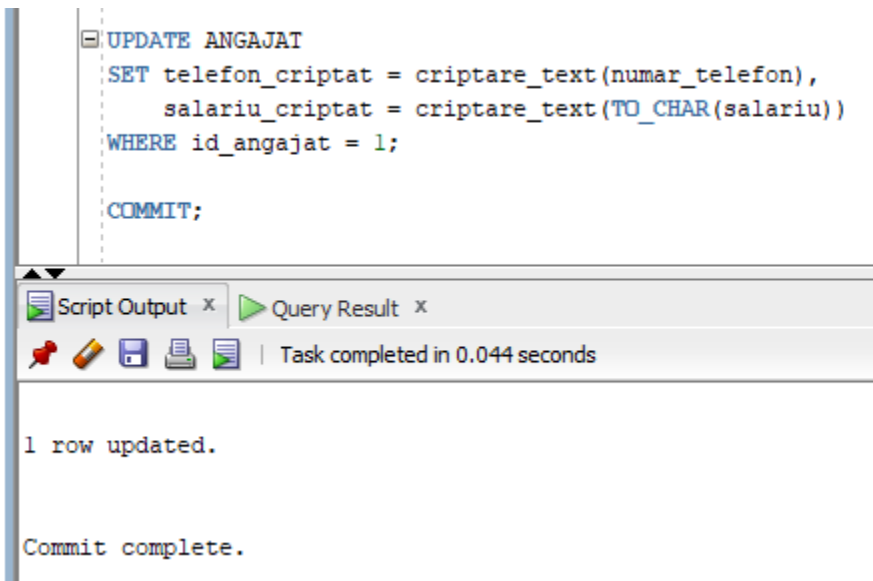
All Rows Fetched: 1 in 0.009 seconds

ID_ANGAJAT	NUME	PRENUME	NUMAR_TELEFON	VARSTA	SPECIALIZARE	EXPERIENTA	SALARIU	COD_ECHIPA	TELEFON_CRIPTAT	SALARIU_CRIPTAT
1	1 Matei	Daniel	0734000001	24 Zidar		2	4500	1 (null)	(null)	

```
--select angajat id 1
SELECT *
FROM ANGAJAT
WHERE id_angajat = 1;

--update angajat id 1 criptare
UPDATE ANGAJAT
SET telefon_criptat = criptare_text(numar_telefon),
    salariu_criptat = criptare_text(TO_CHAR(salariu))
WHERE id_angajat = 1;

COMMIT;
```

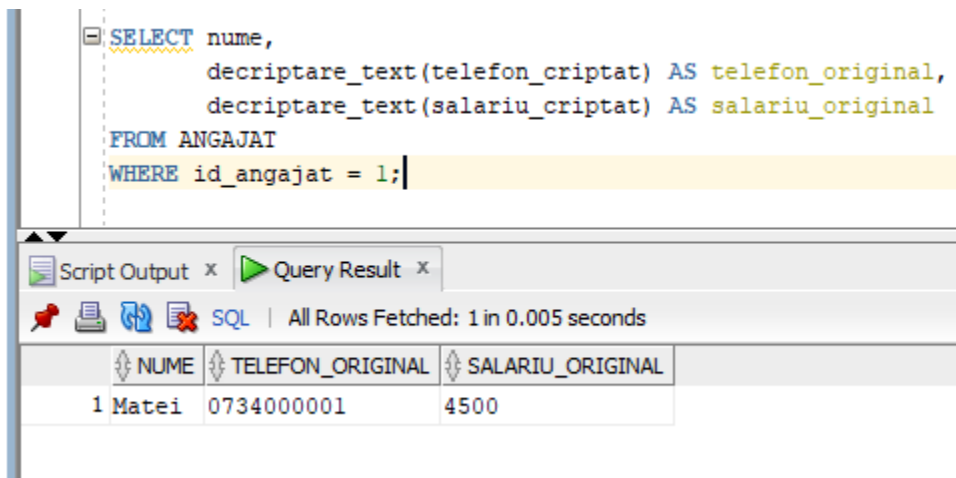


Dovada criptare date sensibile angajat id 1

<pre>SELECT * FROM ANGAJAT WHERE id_angajat = 1;</pre>										
All Rows Fetched: 1 in 0.002 seconds										
ID_ANGAJAT	NUME	PRENUME	NUMAR_TELEFON	VARSTA	SPECIALIZARE	EXPERIENTA	SALARIU	COD_ECHIPA	TELEFON_CRIPTAT	SALARIU_CRIPTAT
1	1 Matei	Daniel	0734000001	24 Zidar		2	4500		1 B670290A969E09EAF11CD6E34A7CAF11 C9C6C7B742631047F770ADF3BC87817D	

Dovada decriptare folosind aceeași cheie folosită la criptare

```
SELECT nume,  
       decryptare_text(telefon_criptat) AS telefon_original,  
       decryptare_text(salariu_criptat) AS salariu_original  
FROM ANGAJAT  
WHERE id_angajat = 1;
```



Se poate observa faptul că numărul de telefon și salariul după decriptare sunt identice cu cele inițiale, deci decriptarea a funcționat conform așteptărilor.

Ulterior, am aplicat algoritmul de criptare pe toate intrările din coloana Anjagat pentru a popula tabelele telefon_criptat și salariu_criptat

```
--populare coloane criptate tabel angajat  
UPDATE ANGAJAT  
SET telefon_criptat = criptare_text(numar_telefon),  
    salariu_criptat = criptare_text(TO_CHAR(salariu));  
  
COMMIT;
```

```
--populare coloane criptate tabel angajat
UPDATE ANGAJAT
SET telefon_criptat = criptare_text(numar_telefon),
    salariu_criptat = criptare_text(TO_CHAR(salariu));

COMMIT;
```

Script Output x Query Result x

Task completed in 0.035 seconds

10 rows updated.

Commit complete.

Script Output x Query Result x

SQL | All Rows Fetched: 10 in 0.003 seconds

ID_ANGAJAT	NUME	PRENUME	NUMAR_TELEFON	VARSTA	SPECIALIZARE	EXPERIENTA	SALARIU	COD_ECHIPA	TELEFON_CRIPTAT	SALARIU_CRIPTAT
1	1 Matei	Daniel	0734000001	24	Zidar	2	4500	1	B670290A969E09EAF11CD6E34A7CAF11	C9C6C7B742631047F770ADF3BC87817D
2	2 Popescu	Andrei	0734000002	29	Electrician	5	5200	2	F0688F7F092A98B32F818630AC777051	F37EE20C3FE106F452A85FC8F60BDAAA
3	3 Ionescu	Maria	0734000003	27	Instalator	4	5000	3	607B1EB137F3C9D7B492C6969FBF8A79	A592F083351C5A8FCB24FDDC75E1E54
4	4 Stan	Cristian	0734000004	31	Finisor	7	5600	4	02F804D8A91879D7C2150CA05813F117	87F90AF1CF7AFFC014DA28EF579A606E
5	5 Voicu	Elena	0734000005	26	Dulgher	3	4700	5	7D33580E1018CD14375938CFF37CC541	251D079866BA7D4B57E8A0FAFA024360
6	6 Marin	Paul	0734000006	28	Operator utilaj	5	5500	6	068120B69402F58E45B55CD9E7D5B19D	F94062A2A809FF48F704487852B3A8AC
7	7 Radu	Alex	0734000007	35	Sudor	10	6500	7	6BE3A6C37F042FF434598377D357A426	BE8E24403C030D2E441983C6C9BC3388
8	8 Enache	Ana	0734000008	30	Arhitect	6	6200	8	4BF637B2D17FE07097B9032037ADA60D	362551E91C0ED1D802B956DDF798CEFO
9	9 Ilie	George	0734000009	25	Topograf	2	4800	9	53D3C7A9D2B7DF1993E0C7E7271ADF08	DDF54C162CAAD93A40DE2AF79C47F485
10	10 Dumitru	Raluca	0734000010	33	Inginer	8	7000	10	890DC161DF056F7D1ECFB99F815B25A0	B71A1599C411F34AC7D33A4EA0A95D07

Într-un scenariu real de producție, valorile originale ar putea fi eliminate sau accesibile doar prin mecanisme controlate de decriptare. În cadrul acestui proiect, coloanele originale au fost păstrate pentru a facilita demonstrarea și verificarea funcționării mecanismului de criptare.

Auditarea activităților asupra bazei de date

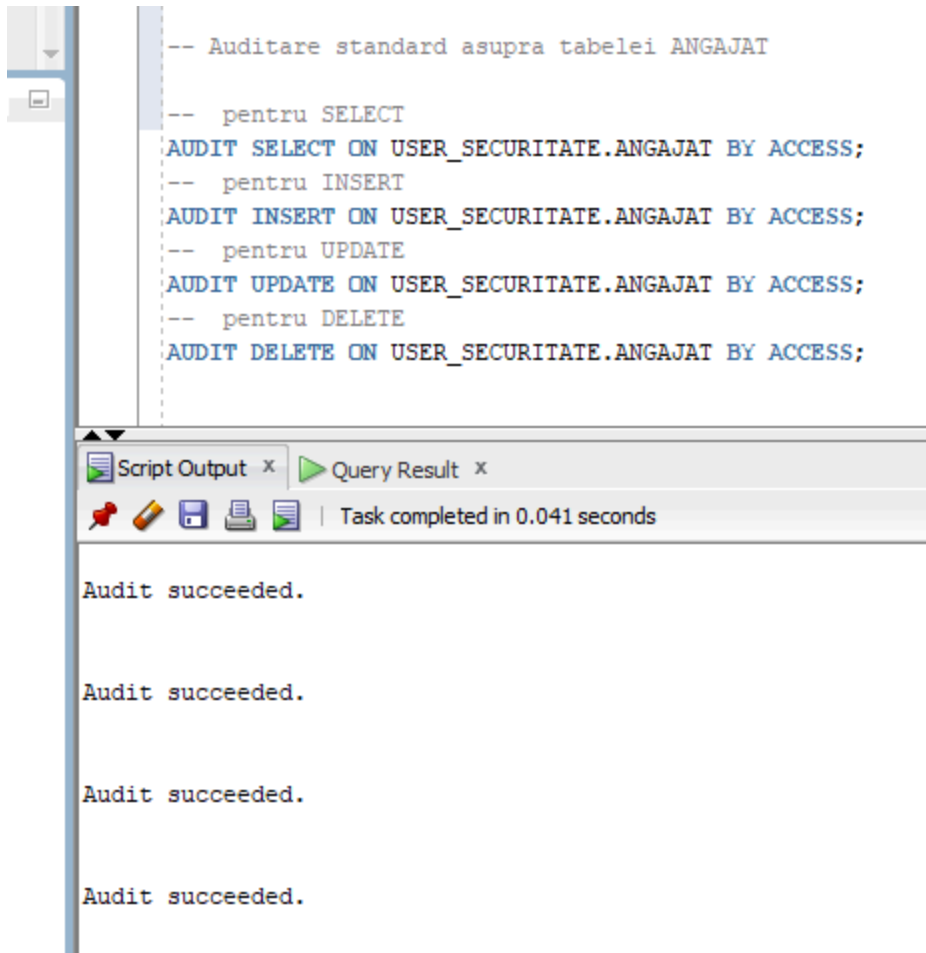
a.Auditare standard

Pentru început, s-a folosit un model standard de auditare din ORACLE asupra tabeli ANGAJAT.

```
--SQL AUDIT

-- Auditare standard asupra tabeli ANGAJAT

-- pentru SELECT
AUDIT SELECT ON USER_SECURITATE.ANGAJAT BY ACCESS;
-- pentru INSERT
AUDIT INSERT ON USER_SECURITATE.ANGAJAT BY ACCESS;
-- pentru UPDATE
AUDIT UPDATE ON USER_SECURITATE.ANGAJAT BY ACCESS;
-- pentru DELETE
AUDIT DELETE ON USER_SECURITATE.ANGAJAT BY ACCESS;
```



Pentru caracterul demonstrativ, am folosit toate operațiunile (INSERT, UPDATE, DELETE, SELECT) asupra tabelii ANGAJAT și am afișat auditul.

```
INSERT INTO ANGAJAT
VALUES
(99, 'Audit', 'Test', '0711111111', 30, 'Electrician', 5, 5000, 1, NULL, NULL);
COMMIT;

UPDATE ANGAJAT
SET SALARIU = 5500
WHERE ID_ANGAJAT = 99;
COMMIT;

DELETE FROM ANGAJAT
WHERE ID_ANGAJAT = 99;
COMMIT;

select * from angajat;
```


Worksheet

Query Builder

```
INSERT INTO ANGAJAT
VALUES (99,'Audit','Test','0711111111',30,'Electrician',5,5000,1,NULL,NULL);
COMMIT;

UPDATE ANGAJAT
SET SALARIU = 5500
WHERE ID_ANGAJAT = 99;
COMMIT;

DELETE FROM ANGAJAT
WHERE ID_ANGAJAT = 99;
COMMIT;

select * from angajat;
```

Script Output x

Query Result x

     | Task completed in 0.018 seconds

1 row inserted.

Commit complete.

Commit complete.

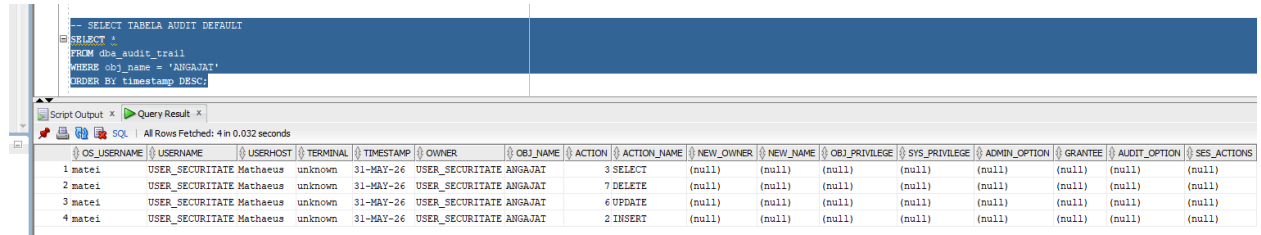
1 row updated.

Commit complete.

1 row deleted.

Commit complete.

```
-- SELECT TABELA AUDIT DEFAULT
SELECT *
FROM dba_audit_trail
WHERE obj_name = 'ANGAJAT'
ORDER BY timestamp DESC;
```



The screenshot shows the SQL Developer interface with a query window containing the SQL statement and a results window displaying the audit trail data. The results window shows 4 rows of data, each representing an audit record for the ANGAJAT table.

OS_USERNAME	USERNAME	USERHOST	TERMINAL	TIMESTAMP	OWNER	OBJ_NAME	ACTION	ACTION_NAME	NEW_OWNER	NEW_NAME	OBJ_PRIVILEGE	SYS_PRIVILEGE	ADMIN_OPTION	GRANTEE	AUDIT_OPTION	SERIALS
1 matei	USER_SECURITATE Mathaeus	unknown		31-MAY-26	USER_SECURITATE ANGAJAT		3 SELECT		(null)	(null)	(null)	(null)	(null)	(null)	(null)	(null)
2 matei	USER_SECURITATE Mathaeus	unknown		31-MAY-26	USER_SECURITATE ANGAJAT		7 DELETE		(null)	(null)	(null)	(null)	(null)	(null)	(null)	(null)
3 matei	USER_SECURITATE Mathaeus	unknown		31-MAY-26	USER_SECURITATE ANGAJAT		6 UPDATE		(null)	(null)	(null)	(null)	(null)	(null)	(null)	(null)
4 matei	USER_SECURITATE Mathaeus	unknown		31-MAY-26	USER_SECURITATE ANGAJAT		2 INSERT		(null)	(null)	(null)	(null)	(null)	(null)	(null)	(null)

b.Trigger-i de auditare

Pentru implementarea auditării personalizate a fost creată tabela AUDIT_ANGAJAT. Aceasta stochează informații despre operațiile efectuate asupra tabelii ANGAJAT, inclusiv utilizatorul care a executat operația, momentul executării și valorile relevante înainte și după modificare.

Principala coloană urmărită este salariul, Trigger-ul având principalul rol de a audita modificările salariale, acestea fiind date sensibile.

```
--creare tabela audit
CREATE TABLE AUDIT_ANGAJAT (
    id_audit NUMBER GENERATED ALWAYS AS IDENTITY,
    utilizator VARCHAR2(30),
    operatie VARCHAR2(10),
    data_operatie DATE,
    id_angajat NUMBER,
    salariu_vechi NUMBER,
    salariu_nou NUMBER,

    CONSTRAINT pk_audit_angajat
    PRIMARY KEY (id_audit)
);
```

```
--auditare cu triggeri

--creare tabela audit
CREATE TABLE AUDIT_ANGAJAT (
    id_audit NUMBER GENERATED ALWAYS AS IDENTITY,
    utilizator VARCHAR2(30),
    operatie VARCHAR2(10),
    data_operatie DATE,
    id_angajat NUMBER,
    salariu_vechi NUMBER,
    salariu_nou NUMBER,

    CONSTRAINT pk_audit_angajat
    PRIMARY KEY (id_audit)
);
```

Script Output x Query Result x

Task completed in 0.038 seconds

Table AUDIT_ANGAJAT created.

În continuare am creat un trigger care urmărește schimbările salariale ale angajaților.

```
--trigger audit schimbari salariale
CREATE OR REPLACE TRIGGER trg_audit_salariu
AFTER INSERT OR UPDATE OR DELETE
ON ANGAJAT
FOR EACH ROW
BEGIN
    -- INSERT
    IF INSERTING THEN
        INSERT INTO AUDIT_ANGAJAT(
            utilizator,
            operatie,
            data_operatie,
            id_angajat,
            salariu_vechi,
            salariu_nou
        )
        VALUES (
            USER,
            'INSERT',
            SYSDATE,
            :NEW.id_angajat,
            NULL,
```

```

        :NEW.salariu
    );
END IF;

-- UPDATE
IF UPDATING THEN
    INSERT INTO AUDIT_ANGAJAT(
        utilizator,
        operatie,
        data_operatie,
        id_angajat,
        salariu_vechi,
        salariu_nou
    )
    VALUES (
        USER,
        'UPDATE',
        SYSDATE,
        :NEW.id_angajat,
        :OLD.salariu,
        :NEW.salariu
    );
END IF;

-- DELETE
IF DELETING THEN
    INSERT INTO AUDIT_ANGAJAT(
        utilizator,
        operatie,
        data_operatie,
        id_angajat,
        salariu_vechi,
        salariu_nou
    )
    VALUES (
        USER,
        'DELETE',
        SYSDATE,
        :OLD.id_angajat,
        :OLD.salariu,
        NULL
    );
END IF;
END;
/

```

Worksheet Query Builder

```
IF UPDATING THEN
    INSERT INTO AUDIT_ANGAJAT (
        utilizator,
        operatie,
        data_operatie,
        id_angajat,
        salariu_vechi,
        salariu_nou
    )
    VALUES (
        USER,
        'UPDATE',
        SYSDATE,
        :NEW.id_angajat,
        :OLD.salariu,
        :NEW.salariu
    );
END IF;

-- DELETE
IF DELETING THEN
    INSERT INTO AUDIT_ANGAJAT (
        utilizator,
        operatie,
        data_operatie,
        id_angajat,
        salariu_vechi,
        salariu_nou
    )
    VALUES (
        USER,
        'DELETE',
        SYSDATE,
        :OLD.id_angajat,
        :OLD.salariu,
        NULL
    );
END IF;
END;
```

Script Output x Query Result x

Task completed in 0.228 seconds

Trigger TRG_AUDIT_SALARIU compiled

Demonstrăm funcționalitatea auditului efectuând operații asupra tabeli angajat, rezultatele se văd în tabela audit creată anterior.

```
--demonstratie functionalitate trigger
INSERT INTO ANGAJAT
VALUES (99,'Audit','Test','0711111111',30,'Electrician',5,5000,1,NULL,NULL);

COMMIT;

UPDATE ANGAJAT
SET salariu = 5500
WHERE id_angajat = 99;

COMMIT;

DELETE FROM ANGAJAT
WHERE id_angajat = 99;

COMMIT;
```

```
--demonstratie functionalitate trigger
INSERT INTO ANGAJAT
VALUES (99,'Audit','Test','0711111111',30,'Electrician',5,5000,1,NULL,NULL);

COMMIT;

UPDATE ANGAJAT
SET salariu = 5500
WHERE id_angajat = 99;

COMMIT;

DELETE FROM ANGAJAT
WHERE id_angajat = 99;

COMMIT;
```

Script Output x Query Result x
Task completed in 0.024 seconds

1 row updated.

Commit complete.

1 row deleted.

Commit complete.

```
--tabela audit
SELECT *
FROM AUDIT_ANGAJAT
ORDER BY id_audit;
```

```
SELECT *
FROM AUDIT_ANGAJAT
ORDER BY id_audit;
```

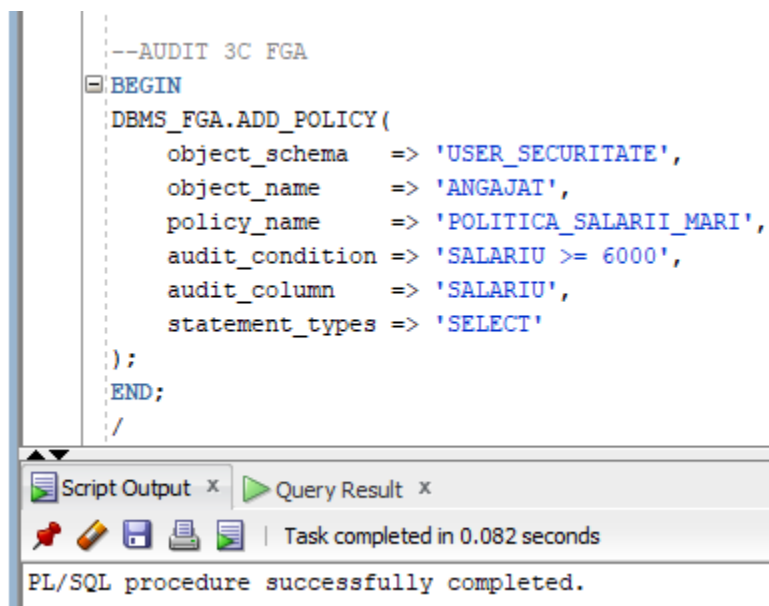
Script Output x Query Result x
SQL | All Rows Fetched: 3 in 0.003 seconds

ID_AUDIT	UTILIZATOR	OPERATIE	DATA_OPERATIE	ID_ANGAJAT	SALARIU_VECHI	SALARIU_NOU
1	1 USER_SECURITATE	INSERT	31-MAY-26	99	(null)	5000
2	2 USER_SECURITATE	UPDATE	31-MAY-26	99	5000	5500
3	3 USER_SECURITATE	DELETE	31-MAY-26	99	5500	(null)

c. Politici de auditare

Pentru implementarea auditării avansate a fost utilizat mecanismul Fine Grained Auditing (FGA) pus la dispoziție de Oracle prin pachetul DBMS_FGA. Spre deosebire de auditarea standard, care înregistrează toate operațiile efectuate asupra unui obiect, FGA permite definirea unor politici care monitorizează doar situații specifice. În cadrul proiectului a fost creată o politică de auditare asupra tabelului ANGAJAT care înregistrează accesările informațiilor salariale ale angajaților cu salarii mai mari sau egale cu 6000. Scopul acestei politici este monitorizarea accesului la informații considerate sensibile.

```
--AUDIT 3C FGA
BEGIN
DBMS_FGA.ADD_POLICY(
    object_schema => 'USER_SECURITATE',
    object_name   => 'ANGAJAT',
    policy_name   => 'POLITICA_SALARII_MARI',
    audit_condition => 'SALARIU >= 6000',
    audit_column  => 'SALARIU',
    statement_types => 'SELECT'
);
END;
/
```



The screenshot displays the Oracle SQL Developer environment. The main window shows the SQL script from the previous block, with the 'BEGIN' statement highlighted. Below the script editor, there are two tabs: 'Script Output' and 'Query Result'. The 'Script Output' tab is active, showing the message 'PL/SQL procedure successfully completed.' at the bottom. Above this message, it indicates 'Task completed in 0.082 seconds'. The interface includes standard icons for saving, running, and other database operations.

```
--AUDIT 3C FGA
BEGIN
DBMS_FGA.ADD_POLICY(
    object_schema => 'USER_SECURITATE',
    object_name   => 'ANGAJAT',
    policy_name   => 'POLITICA_SALARII_MARI',
    audit_condition => 'SALARIU >= 6000',
    audit_column  => 'SALARIU',
    statement_types => 'SELECT'
);
END;
/
```

Script Output x Query Result x

Task completed in 0.082 seconds

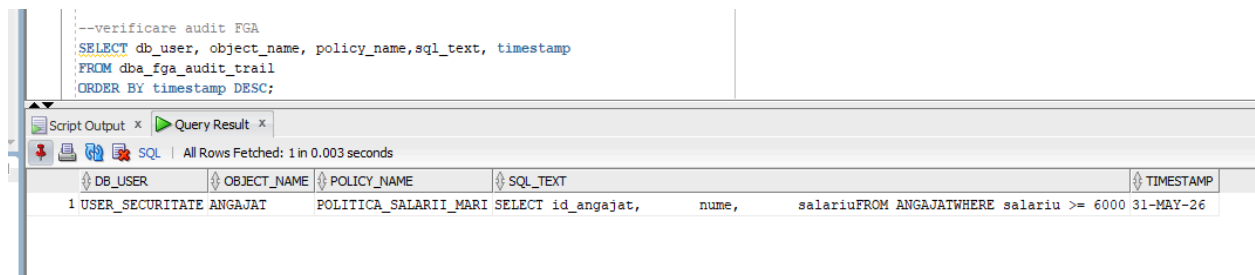
PL/SQL procedure successfully completed.

Verificare, după rularea instrucțiunii următoare cu utilizatorul USER_SECURITATE

```
SELECT id_angajat,nume,salariu  
FROM ANGAJAT  
WHERE salariu >= 6000;
```

S-a updatat tabela de audit

```
--verificare audit FGA  
SELECT db_user, object_name, policy_name,sql_text, timestamp  
FROM dba_fga_audit_trail  
ORDER BY timestamp DESC;
```



DB_USER	OBJECT_NAME	POLICY_NAME	SQL_TEXT	TIMESTAMP
1 USER_SECURITATE	ANGAJAT	POLITICA_SALARII_MARI	SELECT id_angajat, nume, salariu FROM ANGAJAT WHERE salariu >= 6000	31-MAY-26

Gestiunea utilizatorilor unei baze de date și a resurselor computaționale

a.Proiectarea configurației de management a identităților în baza de date (matricile proces-utilizator, entitate-proces, entitate-utilizator)

Pentru administrarea securizată a informațiilor din cadrul firmei de construcții au fost definite mai multe categorii de utilizatori, fiecare având responsabilități și drepturi diferite asupra datelor gestionate de sistem. Separarea responsabilităților permite aplicarea principiului „least privilege”, conform căruia fiecare utilizator primește doar drepturile necesare desfășurării activității sale.

În cadrul proiectului au fost definite următoarele categorii de utilizatori:

- USER_SECURITATE - administratorul bazei de date, responsabil cu gestionarea utilizatorilor, configurarea mecanismelor de securitate, auditarea și administrarea obiectelor bazei de date;
- USER_MANAGER - responsabil pentru gestionarea lucrărilor, clienților, facturilor și recenziilor;

- USER_HR - responsabil pentru administrarea informațiilor referitoare la angajați, șefi și echipe;
- USER_LOGISTICA - responsabil pentru gestionarea utilajelor, colaboratorilor și programărilor aferente acestora.

Userul USER_SECURITATE (practic adminul aplicație) a fost creat la începutul dezvoltării proiectului, ceilalți useri vor fi creați în următorul subcapitol.

Matricea proces - utilizator

Proces	SECURITATE	MANAGER	HR	LOGISTICA
Administrare utilizatori	X	-	-	-
Gestionare angajați	X	-	X	-
Gestionare șefi	X	-	X	-
Gestionare echipe	X	-	X	-
Gestionare lucrări	X	X	-	-
Gestionare clienți	X	X	-	-
Gestionare facturi	X	X	-	-
Gestionare recenzii	X	X	-	-
Gestionare utilaje	X	-	-	X
Gestionare colaboratori	X	-	-	X
Gestionare programări utilaje	X	-	-	X
Consultare audit	X	-	-	-

Matricea entitate - proces

Proces	Entități implicate
Administrare utilizatori	USERI, ROLURI
Gestionare angajați	ANGAJAT
Gestionare șefi	SEF
Gestionare echipe	ECHIPA
Gestionare lucrări	LUCRARE, LOCATIE
Gestionare clienți	CLIENT
Gestionare facturi	FACTURA, OFERTA
Gestionare recenzii	RECENZIE
Gestionare utilaje	UTILAJ
Gestionare colaboratori	COLABORATORI, SELECTIE_COLABORATORI
Gestionare programări utilaje	PROGRAMARE_UTILAJE
Consultare audit	AUDIT_ANGAJAT, DBA_AUDIT_TRAIL

Matricea entitate - proces

Entitate	SECURITATE	MANAGER	HR	LOGISTICA
CLIENT	CRUD	CRUD	-	-
LUCRARE	CRUD	CRUD	-	-
LOCATIE	CRUD	CRUD	-	-
FACTURA	CRUD	CRUD	-	-
OFERTA	CRUD	CRUD	-	-
RECENZIE	CRUD	CRUD	-	-
ANGAJAT	CRUD	-	CRUD	-
SEF	CRUD	-	CRUD	-
ECHIPA	CRUD	-	CRUD	-
UTILAJ	CRUD	-	-	CRUD
PROGRAMARE_UTILAJE	CRUD	-	-	CRUD
COLABORATORI	CRUD	-	-	CRUD
SELECTIE_COLABORATORI	CRUD	-	-	CRUD
AUDIT_ANGAJAT	CRUD	-	-	-

a.Implementarea configurației de management a identităților în baza de date

Am creat utilizatori corespunzători fiecărui rol funcțional din firmă, în conformitate cu matricea entitate-utilizator din proiect. Rolul de admin (USER_SECURITATE) a fost creat la începutul proiectului și i-au fost acordate toate drepturile necesare. Creare userilor a fost făcută din utilizatorul pentru sistem (sys).

```
-- Creare utilizator pentru managementul lucrărilor și clienților
```

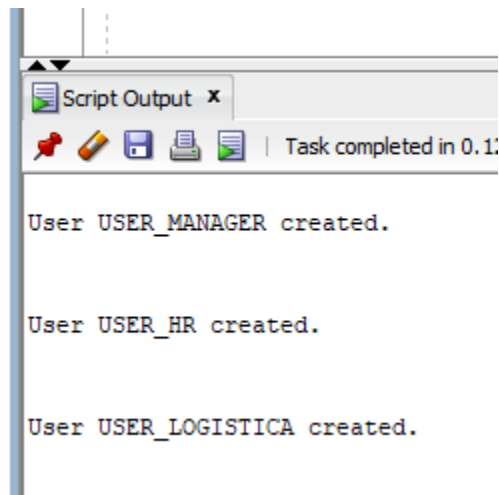
```
CREATE USER USER_MANAGER  
IDENTIFIED BY 1234;
```

```
-- Creare utilizator pentru departamentul HR
```

```
CREATE USER USER_HR  
IDENTIFIED BY 1234;
```

```
-- Creare utilizator pentru departamentul Logistică
```

```
CREATE USER USER_LOGISTICA  
IDENTIFIED BY 1234;
```



```
-- Acordare drept de conectare
```

```
GRANT CONNECT TO USER_MANAGER;
```

```
GRANT CONNECT TO USER_HR;
```

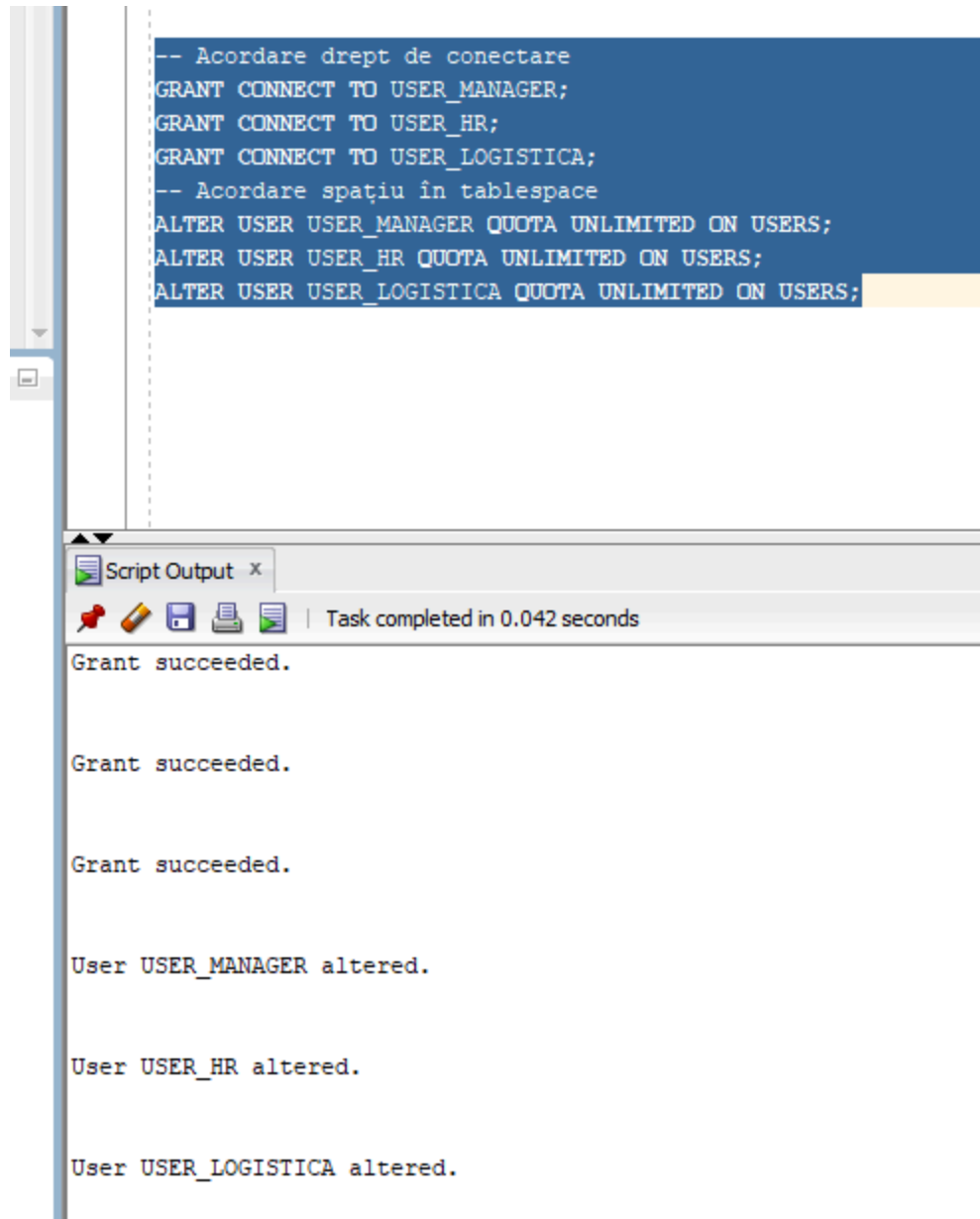
```
GRANT CONNECT TO USER_LOGISTICA;
```

```
-- Acordare spațiu în tablespace
```

```
ALTER USER USER_MANAGER QUOTA UNLIMITED ON USERS;
```

```
ALTER USER USER_HR QUOTA UNLIMITED ON USERS;
```

```
ALTER USER USER_LOGISTICA QUOTA UNLIMITED ON USERS;
```



```
-- Acordare drept de conectare
GRANT CONNECT TO USER_MANAGER;
GRANT CONNECT TO USER_HR;
GRANT CONNECT TO USER_LOGISTICA;
-- Acordare spațiu în tablespace
ALTER USER USER_MANAGER QUOTA UNLIMITED ON USERS;
ALTER USER USER_HR QUOTA UNLIMITED ON USERS;
ALTER USER USER_LOGISTICA QUOTA UNLIMITED ON USERS;
```

Script Output x

Task completed in 0.042 seconds

Grant succeeded.

Grant succeeded.

Grant succeeded.

User USER_MANAGER altered.

User USER_HR altered.

User USER_LOGISTICA altered.

Acordarea drepturilor precizate în punctele anterioare fiecărui user.

```
-- USER_MANAGER gestionează CLIENT, LUCRARE, LOCATIE, FACTURA, OFERTA, RECENZIE
GRANT SELECT, INSERT, UPDATE, DELETE ON USER_SECURITATE.CLIENT TO USER_MANAGER;
GRANT SELECT, INSERT, UPDATE, DELETE ON USER_SECURITATE.LUCRARE TO
USER_MANAGER;
GRANT SELECT, INSERT, UPDATE, DELETE ON USER_SECURITATE.LOCATIE TO
USER_MANAGER;
GRANT SELECT, INSERT, UPDATE, DELETE ON USER_SECURITATE.FACTURA TO
USER_MANAGER;
GRANT SELECT, INSERT, UPDATE, DELETE ON USER_SECURITATE.OFERTA TO USER_MANAGER;
```

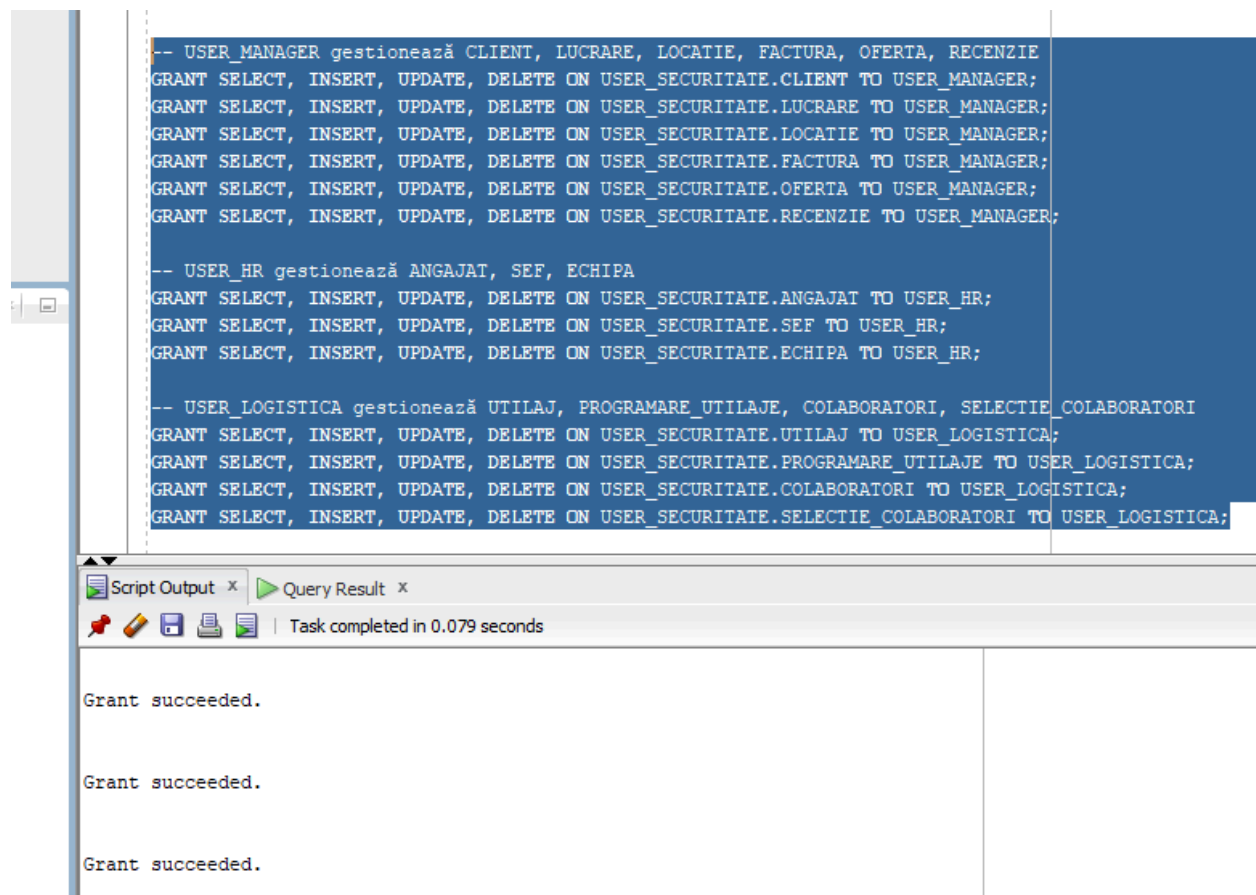
```

GRANT SELECT, INSERT, UPDATE, DELETE ON USER_SECURITATE.RECENZIE TO
USER_MANAGER;

-- USER_HR gestionează ANGAJAT, SEF, ECHIPA
GRANT SELECT, INSERT, UPDATE, DELETE ON USER_SECURITATE.ANGAJAT TO USER_HR;
GRANT SELECT, INSERT, UPDATE, DELETE ON USER_SECURITATE.SEF TO USER_HR;
GRANT SELECT, INSERT, UPDATE, DELETE ON USER_SECURITATE.ECHIPA TO USER_HR;

-- USER_LOGISTICA gestionează UTILAJ, PROGRAMARE_UTILAJE, COLABORATORI,
SELECTIE_COLABORATORI
GRANT SELECT, INSERT, UPDATE, DELETE ON USER_SECURITATE.UTILAJ TO
USER_LOGISTICA;
GRANT SELECT, INSERT, UPDATE, DELETE ON USER_SECURITATE.PROGRAMARE_UTILAJE TO
USER_LOGISTICA;
GRANT SELECT, INSERT, UPDATE, DELETE ON USER_SECURITATE.COLABORATORI TO
USER_LOGISTICA;
GRANT SELECT, INSERT, UPDATE, DELETE ON USER_SECURITATE.SELECTIE_COLABORATORI
TO USER_LOGISTICA;

```



```

-- USER_MANAGER gestionează CLIENT, LUCRARE, LOCATIE, FACTURA, OFERTA, RECENZIE
GRANT SELECT, INSERT, UPDATE, DELETE ON USER_SECURITATE.CLIENT TO USER_MANAGER;
GRANT SELECT, INSERT, UPDATE, DELETE ON USER_SECURITATE.LUCRARE TO USER_MANAGER;
GRANT SELECT, INSERT, UPDATE, DELETE ON USER_SECURITATE.LOCATIE TO USER_MANAGER;
GRANT SELECT, INSERT, UPDATE, DELETE ON USER_SECURITATE.FACTURA TO USER_MANAGER;
GRANT SELECT, INSERT, UPDATE, DELETE ON USER_SECURITATE.OFERTA TO USER_MANAGER;
GRANT SELECT, INSERT, UPDATE, DELETE ON USER_SECURITATE.RECENZIE TO USER_MANAGER;

-- USER_HR gestionează ANGAJAT, SEF, ECHIPA
GRANT SELECT, INSERT, UPDATE, DELETE ON USER_SECURITATE.ANGAJAT TO USER_HR;
GRANT SELECT, INSERT, UPDATE, DELETE ON USER_SECURITATE.SEF TO USER_HR;
GRANT SELECT, INSERT, UPDATE, DELETE ON USER_SECURITATE.ECHIPA TO USER_HR;

-- USER_LOGISTICA gestionează UTILAJ, PROGRAMARE_UTILAJE, COLABORATORI, SELECTIE_COLABORATORI
GRANT SELECT, INSERT, UPDATE, DELETE ON USER_SECURITATE.UTILAJ TO USER_LOGISTICA;
GRANT SELECT, INSERT, UPDATE, DELETE ON USER_SECURITATE.PROGRAMARE_UTILAJE TO USER_LOGISTICA;
GRANT SELECT, INSERT, UPDATE, DELETE ON USER_SECURITATE.COLABORATORI TO USER_LOGISTICA;
GRANT SELECT, INSERT, UPDATE, DELETE ON USER_SECURITATE.SELECTIE_COLABORATORI TO USER_LOGISTICA;

```

Script Output x Query Result x

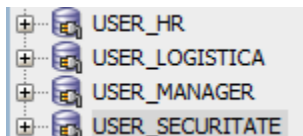
Task completed in 0.079 seconds

Grant succeeded.

Grant succeeded.

Grant succeeded.

Mai jos putem observa conexiunile specifice fiecărui user



Privilegii și roluri

a.Privilegii sistem și obiect

În Oracle, privilegiile se împart în două categorii:

- privilegii de sistem, care permit executarea unor operații generale asupra bazei de date
- privilegii asupra obiectelor, care permit accesul controlat la tabele, vizualizări sau alte obiecte aparținând unui utilizator.

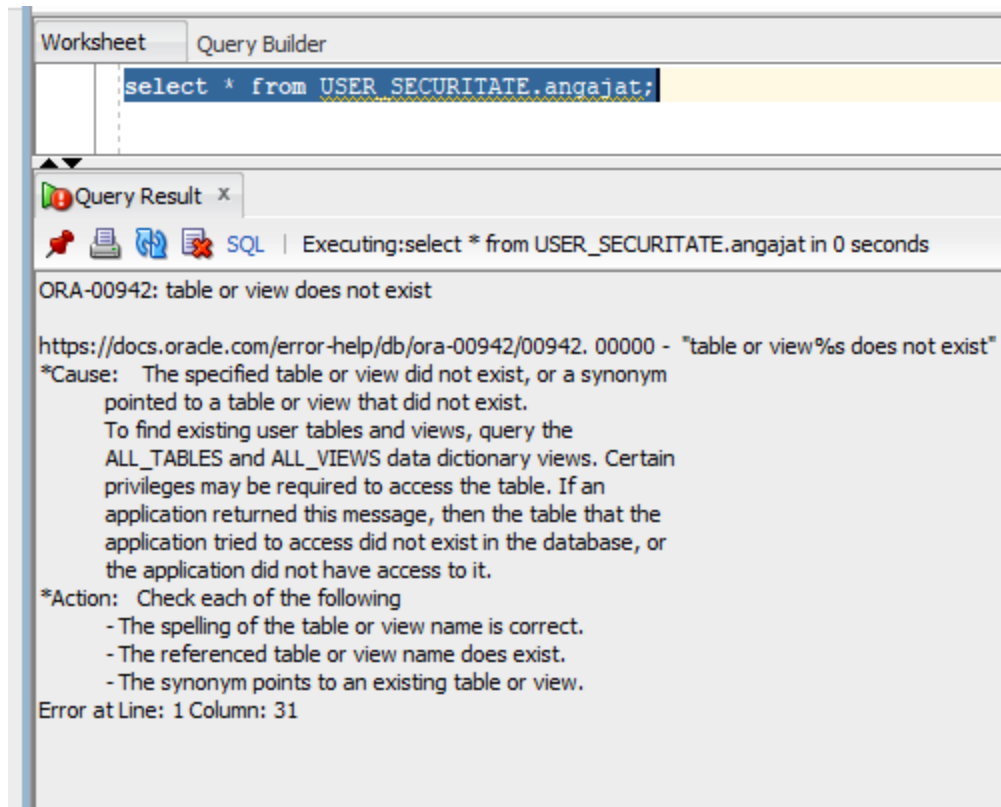
În cadrul proiectului au fost utilizate ambele categorii de privilegii.

Utilizatorilor USER_SECURITATE(practic adminul), USER_MANAGER, USER_HR și USER_LOGISTICA le-a fost acordat privilegiul de sistem CONNECT, necesar autentificării și inițierii unei sesiuni de lucru în baza de date.

Pentru implementarea controlului accesului la date au fost utilizate privilegii asupra obiectelor. Astfel, fiecare utilizator are acces strict la informațiile de care are nevoie pentru rolul pe care îl are.

Demonstrație:

Interogare ANGAJAT din USER_MANAGER



Interogare ANGAJAT din USER_HR

ID_ANGAJAT	NUME	PRENUME	NUMAR_TELEFON	VARSTA	SPECIALIZARE	EXPERIENTA	SALARIU	COD_ECHIPA	TELEFON_CRIPTAT	SALARIU_CRIPTAT
1	1 Matei	Daniel	0734000001	24	Zider	2	4500	1 B670290A969E09EAF11CD6E34A7CAF11	C9C6C7B742631047F770ADF3BC87817D	
2	2 Popescu	Andrei	0734000002	29	Electrician	5	5200	2 F0688F7F092A98B32F818630AC777051	F37EE20C3FE106F452A85FC8F60BDAAA	
3	3 Ionescu	Maria	0734000003	27	Instalator	4	5000	3 607B1EB137F3C9D7B492C6969FBF8A79	A592F083351C5A8FCB24FDDDEC75E1E54	
4	4 Stan	Cristian	0734000004	31	Finisor	7	5600	4 02F804D8A91879D7C2150CA05813F117	87F90AF1CF7AFFC014DA28EF579A606E	
5	5 Voicu	Elena	0734000005	26	Dulgher	3	4700	5 7D33580E1018CD14375938CFF37CC541	251D079866BA7D4B57E8A0FAFA024360	
6	6 Marin	Paul	0734000006	28	Operator utilaj	5	5500	6 068120B69402F58E49B55CD9E7D5B19D	F94062A2A809FF48F704487852B3A8AC	
7	7 Radu	Alex	0734000007	35	Sudor	10	6500	7 6BE3A6C37F042FF434598377D357A426	BEBE24403C030D2E441983C6C9BC388	
8	8 Enache	Ana	0734000008	30	Arhitect	6	6200	8 4BF637B2D17FE07097B9032037ADA60D	362551E91C0ED1D02B956DDF799CEFO	
9	9 Ilie	George	0734000009	25	Topograf	2	4800	9 53D3C7A9D2B7DF1993E0C7E7271ADFC8	DDF54C162CAAD93A40DE2AF79C47F485	
10	10 Dumitru	Raluca	0734000010	33	Inginer	8	7000	10 890DC161DF056F7D1ECFB99F815B25A0	B71A1599C411F34AC7D33A4EA0A95D07	

Pe lângă privilegiul de conectare la baza de date, utilizatorilor definiți în cadrul proiectului le-a fost alocată și o cotă de spațiu în tablespace-ul USERS, utilizând opțiunea QUOTA UNLIMITED ON USERS. Prin această configurare, utilizatorii pot crea și gestiona obiecte proprii în schema lor, fără restricții privind spațiul disponibil în tablespace-ul asociat.

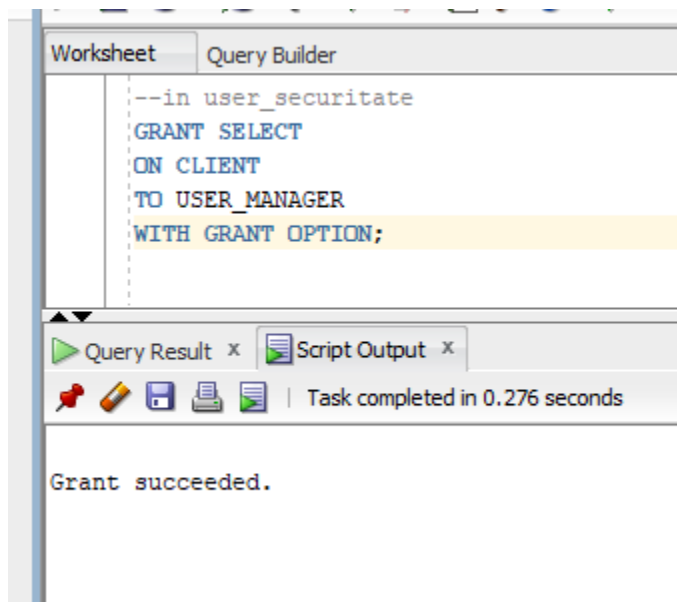
b. Ierarhii de privilegii

În Oracle, privilegiile pot fi propagate între utilizatori prin utilizarea opțiunilor WITH GRANT OPTION și WITH ADMIN OPTION. Aceste mecanisme permit unui utilizator care a primit un privilegiu să îl acorde la rândul său altor utilizatori, realizând astfel o ierarhie a privilegiilor.

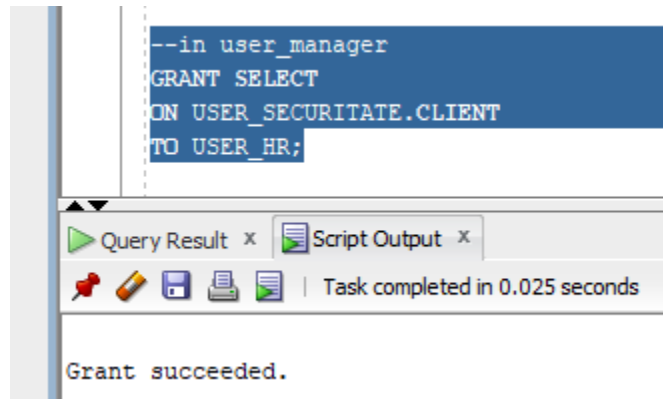
În cadrul proiectului, cu scop demonstrativ, am decis să acordăm drepturi pe tabela CLIENT și utilizatorului USER_HR pentru a crea o ierarhie.

1. USER_SECURITATE acordă privilegiul către USER_MANAGER.
2. USER_MANAGER transmite privilegiul către USER_HR.
3. USER_HR poate consulta tabela CLIENT.

```
--in user_securitate  
GRANT SELECT  
ON CLIENT  
TO USER_MANAGER  
WITH GRANT OPTION;
```



```
--in user_manager
GRANT SELECT
ON USER_SECURITATE.CLIENT
TO USER_HR;
```



Din USER_HR avem acces la tabela CLIENT.

```
--IN USER_HR
select * from user_securitate.client;
```

The screenshot shows a SQL query execution window with a yellow header bar containing the following text:

```
--IN USER_HR
select * from user_securitate.client;
```

Below the header bar, there is a status bar indicating "All Rows Fetched: 10 in 0.002 seconds". The main area of the window displays a table with 10 rows and 6 columns. The columns are labeled: ID_CLIENT, NUME, PRENUME, NUMAR_TELEFON, and NR_LUCRARI_ANTERIOARE. The data is as follows:

	ID_CLIENT	NUME	PRENUME	NUMAR_TELEFON	NR_LUCRARI_ANTERIOARE
1	1	Popescu	Andrei	0712000001	2
2	2	Ionescu	Maria	0712000002	0
3	3	Dumitru	Alex	0712000003	1
4	4	Georgescu	Elena	0712000004	4
5	5	Stan	Mihai	0712000005	3
6	6	Radu	Cristina	0712000006	0
7	7	Voicu	Daniel	0712000007	5
8	8	Marin	Ana	0712000008	2
9	9	Enache	Paul	0712000009	1
10	10	Ilie	Raluca	0712000010	6

Prin urmare, a fost demonstrată propagarea privilegiilor prin intermediul opțiunii WITH GRANT OPTION și realizarea unei ierarhii de acces de forma USER_SECURITATE -> USER_MANAGER -> USER_HR

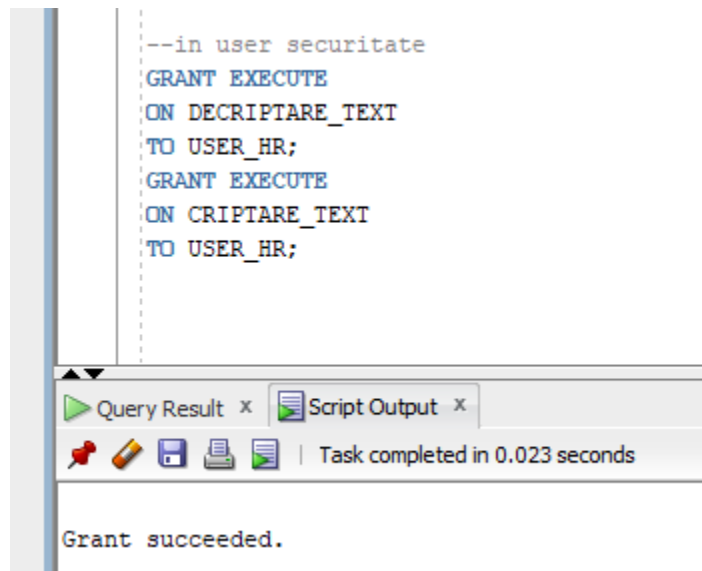
c.Ierarhii de privilegii

În cadrul proiectului au fost definite funcțiile CRIPTARE_TEXT și DECRYPTARE_TEXT pentru protejarea informațiilor sensibile din tabela ANGAJAT.

Funcțiile PL/SQL CRIPTARE_TEXT și DECRYPTARE_TEXT reprezintă obiecte dependente ale bazei de date. Pentru utilizarea acestora de către alți utilizatori este necesară acordarea explicită a privilegiului EXECUTE asupra funcțiilor respective.

USER_HR este singurul user, pe lângă USER_SECURITATE (admin) care are acces la funcțiile de criptare și decriptare pentru datele sensibile ale angajaților.

```
--in user securitate
GRANT EXECUTE
ON DECRYPTARE_TEXT
TO USER_HR;
GRANT EXECUTE
ON CRIPTARE_TEXT
TO USER_HR;
```



```
--in user_hr
SELECT
    id_angajat,
    USER_SECURITATE.DECRYPTARE_TEXT(telefon_criptat) AS telefon_decriptat,
    telefon_criptat
FROM USER_SECURITATE.ANGAJAT;
```

```
--in user_hr
SELECT
    id_angajat,
    USER_SECURITATE.DECRIPTARE_TEXT(telefon_criptat) AS telefon_decriptat,
    telefon_criptat
FROM USER_SECURITATE.ANGAJAT;
```

Query Result x

SQL | All Rows Fetched: 10 in 0.071 seconds

	ID_ANGAJAT	TELEFON_DECRIPATAT	TELEFON_CRIPTAT
1	1	0734000001	B670290A969E09EAF11CD6E34A7CAF11
2	2	0734000002	F0688F7F092A98B32F818630AC777051
3	3	0734000003	607B1EB137F3C9D7B492C6969FBF8A79
4	4	0734000004	02F804D8A91879D7C2150CA05813F117
5	5	0734000005	7D33580E1018CD14375938CFF37CC541
6	6	0734000006	068120B69402F58E49B55CD9E7D5B19D
7	7	0734000007	6BE3A6C37F042FF434598377D357A426
8	8	0734000008	4BF637B2D17FE07097B9032037ADA60D
9	9	0734000009	53D3C7A9D2B7DF1993E0C7E7271ADFC8
10	10	0734000010	890DC161DF056F7D1ECFB99F815B25A0